

From Traditional RAG to Agentic RAG

A complete walkthrough of the Company Policy Assistant
Built with ChromaDB, LangGraph, and GitHub Models

June 2026

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. Instead of asking an AI model to answer from its general training data (which may be outdated or wrong), we first search our own documents for relevant information, then hand that information to the AI along with the question. This way, the AI answers based on YOUR data, not its imagination.

Think of it like this:

- Without RAG: You ask a stranger a question about your company's leave policy. They guess.
- With RAG: You hand the stranger your company's policy handbook, highlight the relevant pages, and THEN ask the question. They answer based on what's actually written.

2. How Documents Get Ingested (ingest.py)

Before we can search our documents, we need to prepare them. This is what the ingestion step does. It runs once, before the app starts.

Step 1: Read the Documents

The system reads all files from the documents/ folder. It supports plain text (.txt) and PDF (.pdf) files. Each file is read and its full text is extracted.

Step 2: Split into Chunks

A full document can be thousands of words long. Searching through entire documents is slow and imprecise. So we split each document into smaller pieces called 'chunks' -- roughly 500 words each, with a 50-word overlap between consecutive chunks so we don't lose context at the boundaries.

Example: A 1500-word document becomes roughly 3-4 chunks.

Step 3: Convert to Vectors (Embeddings)

Here's the key idea: we convert each text chunk into a list of numbers called a 'vector' or 'embedding'. This is done by a Sentence Transformer model (BAAI/bge-base-en-v1.5). These numbers capture the MEANING of the text -- so chunks about similar topics will have similar numbers, even if they use different words.

Step 4: Store in ChromaDB

ChromaDB is a vector database. It stores each chunk along with its vector and metadata (like which file it came from). Later, when a user asks a question, we convert the question into a vector too, and ChromaDB finds the chunks whose vectors are closest -- meaning the chunks most semantically similar to the question.

The ingested data is saved to disk in the chroma_db/ folder, so it persists across restarts.

Ingestion Flow

documents/		Chunks		Vectors
leave_policy.txt	-->	[chunk1, chunk2, ...]	-->	[0.12, -0.34, ...]
remote_work.txt	-->	[chunk1, chunk2, ...]	-->	[0.56, 0.78, ...]
expense_policy.txt	-->	[chunk1, chunk2, ...]	-->	[-0.23, 0.45, ...]

All stored in ChromaDB (chroma_db/)

3. Traditional RAG -- How It Worked (rag.py)

Our first version used a simple, linear pipeline. When a user types a question in the Streamlit chat UI, here's exactly what happened:

Step 1: Retrieve

The user's question is converted into a vector (using the same embedding model). ChromaDB searches for the 5 closest matching chunks. These are our 'context' documents.

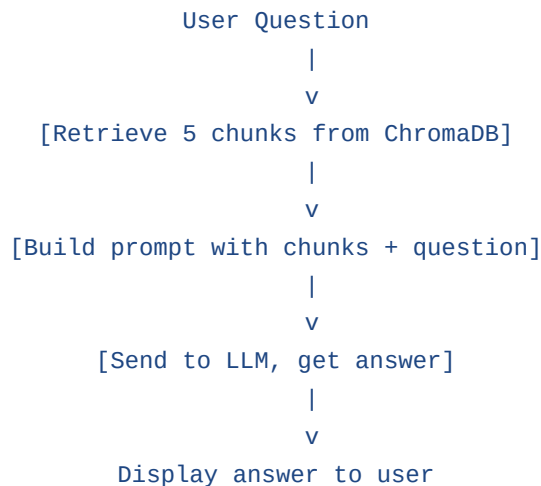
Step 2: Build Prompt

The retrieved chunks are formatted and injected into a prompt along with the user's question. A system prompt tells the AI to ONLY answer from the provided context, cite sources, and never make things up.

Step 3: Generate

The prompt is sent to the LLM (GPT-4o-mini via GitHub Models). The AI reads the context and generates an answer. The response is streamed back to the chat UI token by token.

Traditional RAG Flow



4. The Problem with Traditional RAG

The traditional approach works well for simple, direct questions like:

- "What is the remote work policy?" -- Easy. The chunks about remote work are retrieved and answered.
- "How many sick leaves do I get?" -- Direct match. Works great.

But it struggles with:

Problem 1: Broad or Vague Queries

A question like "List all dollar amounts in the policies" doesn't match any specific policy topic well. The vector search might return chunks that are only loosely related, and the AI either gives an incomplete answer or says it doesn't know.

Problem 2: No Quality Check

Traditional RAG blindly trusts whatever chunks come back from the search. If the search returns irrelevant chunks (maybe the question was phrased in an unusual way), the AI tries to answer from bad context -- leading to wrong or unhelpful answers.

Problem 3: One Shot, No Retry

You get one attempt at retrieval. If the wording of the question doesn't match the wording in the documents, you're stuck with poor results. There's no mechanism to rephrase and try again.

In short: Traditional RAG is a one-way street. It retrieves once, generates once, and hopes for the best. There's no intelligence in the retrieval process itself.

5. Agentic RAG -- The Solution (agentic_rag.py)

Agentic RAG adds a brain to the retrieval process. Instead of a straight pipeline, we build a GRAPH -- a set of steps with decision points where the system can choose what to do next, including looping back to try again.

We use LangGraph to define this graph. Here are the four nodes (steps):

Node 1: Retrieve

Same as before -- query ChromaDB and get the top 5 matching chunks. But now, instead of going straight to the LLM, the chunks go through a quality check first.

Node 2: Grade Documents

This is the key addition. For each retrieved chunk, we ask the LLM: 'Is this document actually relevant to the user's question?' The LLM evaluates each chunk individually and returns a yes/no verdict. Only chunks marked as relevant are kept.

The grader is intentionally lenient -- if a chunk is even partially related, it's kept. This prevents the system from being too aggressive in filtering.

Node 3: Rewrite Query

If the grading step finds that fewer than 1 relevant document was retrieved, something went wrong with the search. Instead of giving up, the system asks the LLM to REWRITE the question -- make it more specific, use different keywords, break it down. Then it loops back to retrieve again with the new query.

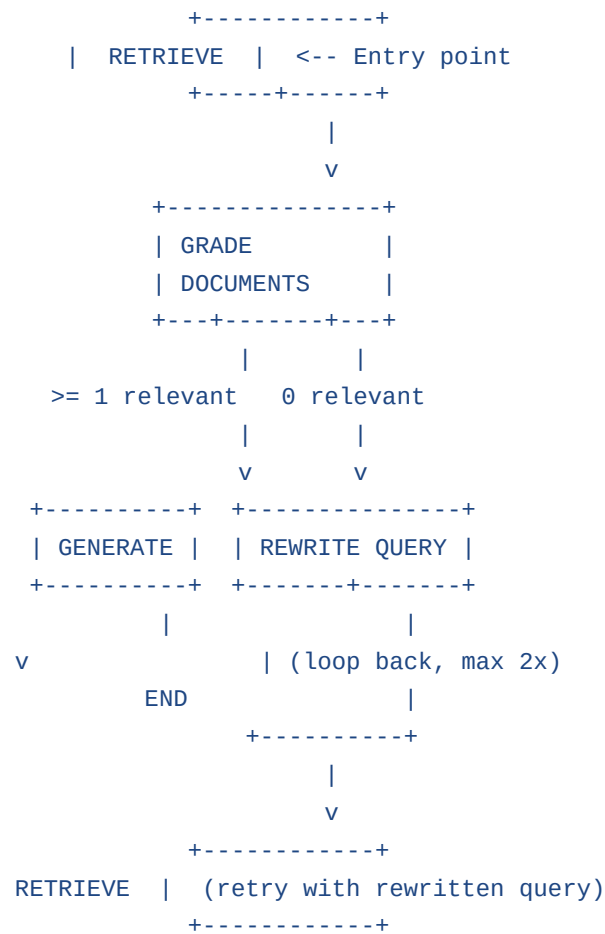
This can happen up to 2 times. After 2 retries, it generates with whatever it has.

Node 4: Generate

Once we have verified relevant documents, the system builds the prompt (same system prompt as before) and sends it to the LLM for the final answer. The response is streamed to the UI.

6. The LangGraph Structure

Here's how the graph is wired together:



The conditional edge after grading is the decision point:

- If at least 1 relevant document exists --> go to Generate
- If 0 relevant documents AND retries < 2 --> go to Rewrite Query
- If 0 relevant documents AND retries >= 2 --> go to Generate anyway (best effort)

7. What Happens When You Ask a Question

Let's trace through a real example. You type in the UI:

```
"Can a part-time intern work remotely while on probation?"
```

Step 1: Streamlit captures the input

The app.py file receives your message, adds it to the chat history, and calls `stream_ask()` from `agentic_rag.py`.

Step 2: Retrieve

Your question is vectorized and sent to ChromaDB. It returns 5 chunks -- maybe some about remote work, some about leave policy, some about internship terms.

Step 3: Grade Documents

The LLM checks each chunk: 'Is this relevant to part-time interns working remotely during probation?' Maybe 3 out of 5 pass. The other 2 are discarded.

Step 4: Decision

3 relevant documents ≥ 1 threshold. Decision: go to Generate (no rewrite needed).

Step 5: Generate

The 3 verified chunks are formatted into the prompt. The LLM reads them and generates a specific answer about remote work eligibility for part-time interns on probation.

Step 6: Display

The answer streams into the chat UI. The sidebar shows the agent trace -- you can see exactly which steps were taken: retrieve, grade (3/5 relevant), generate.

Now imagine a trickier query that DOES trigger a rewrite:

```
"List all dollar amounts mentioned in the company policies"
```

1. Retrieve: Gets 5 chunks, but they're about general policy topics, not specifically about amounts.
2. Grade: The grader finds 0 relevant chunks (none specifically list dollar amounts).
3. Decision: 0 relevant docs, retries = 0 --> Rewrite Query.
4. Rewrite: LLM rephrases to something like 'expense reimbursement limits and monetary amounts in policies'.
5. Retrieve (again): New query gets better chunks -- ones mentioning specific dollar figures.
6. Grade (again): 3 out of 5 are relevant.
7. Decision: 3 ≥ 1 --> Generate.
8. Generate: Produces a comprehensive answer listing the dollar amounts from the policies.

8. The Agent Trace (Sidebar)

Every step the agent takes is logged in a trace list. After each query, the sidebar shows exactly what happened. This makes the system transparent and debuggable.

Example trace for a simple query:

```
retrieve(query='What is the remote work policy?')
grade_documents(4/5 relevant)
generate(streamed)
```

Example trace for a query that needed rewriting:

```
retrieve(query='list all dollar amounts')
grade_documents(0/5 relevant)
rewrite_query('expense reimbursement limits and monetary amounts')
retrieve(query='expense reimbursement limits and monetary amounts')
grade_documents(3/5 relevant)
generate(streamed)
```

9. Traditional RAG vs Agentic RAG -- Summary

Aspect	Traditional RAG	Agentic RAG
Retrieval	Single attempt	Retrieves + retries if needed
Quality Check	None	LLM grades each document
Query Handling	Uses query as-is	Rewrites if results are poor
Flow	Linear (one-way)	Graph with loops
Transparency	Black box	Full agent trace visible
Broad Queries	Often fails	Adapts via rewriting
Complexity	Simple, fast	More LLM calls, smarter

10. Tech Stack

- Streamlit -- Chat UI for interacting with the assistant
- ChromaDB -- Vector database for storing and searching document embeddings
- Sentence Transformers (BAAI/bge-base-en-v1.5) -- Converts text to vectors
- OpenAI SDK (GitHub Models / GPT-4o-mini) -- LLM for grading, rewriting, and answering
- LangGraph -- Framework for building the agentic graph with nodes, edges, and loops
- Python + uv -- Runtime and package management